There Must Be Some Kind of Mistake: Distinguishing Errors from Uncertainty in Type Checking

A recent paper by Garby, Bahr, and Hutton proposes a relational specification for type checkers, where types computed statically are related to the types of computational results. The relation in question makes type errors smaller than non-error types. This paper refines this relation by introducing explicit types for terms which are guaranteed to exhibit runtime errors of various kinds. These error types are distinct from a type representing uncertainty about the term's runtime behavior. The two kinds of errors considered are finite failure and divergence. To manage the complexity induced by the richer partial order on types, the paper uses a monadic type checker, which is then related to a fuel-bounded monadic interpreter. Proof complexity is also managed monadically, using a binary indexed monad to relate typing and evaluation. The verification is carried out in Agda.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; • **Theory of computation** → *Program verification*; **Type theory**.

Additional Key Words and Phrases: type checking, nontermination, type errors, monads

ACM Reference Format:
. 2026. There Must Be Some Kind of Mistake: Distinguishing Errors from Uncertainty in Type Checking. *J. ACM* 37, 4, Article 111 (August 2026), 19 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Garby et al. [3] present the derivation of a type checker for a simple expression language, in the style of program calculation. With this approach, one tries to prove a specification about code that has yet to be determined, and discovers the definitions (or constraints on them) necessary in the course of attempting to complete the proof. Their specification states that the type computed by the type checker `texp` for an *Expr e* is an approximation to the true type of *e*. The true type is the type computed (by a function *tval*) for the value of *e*, as produced by an evaluator *eval*. The approximation ($\leq$) states that the *ERROR* type returned by the type checker to signal a type error approximates any of the types for values. With the types shown in Figure 1 (following the authors in writing Haskell with unicode), their specification is

$$texp\ e \leq tval\ (eval\ e)$$

While acknowledging that usually one would implement the evaluator monadically to handle runtime errors, they opt to use simpler typing, and include `Error` as a `Value`.

In this paper, we build on the prior work of Garby et al. in the following ways:

(1) We extend the ordering on types to distinguish known errors from uncertainty. So we now have types `UNKNOWN` and `FAIL`. If an expression is of type `FAIL`, that means that it is guaranteed to produce a runtime error. `UNKNOWN` represents a term whose runtime behavior cannot be assessed by the type checker.

Author's address:

ACM 0004-5411/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

```
data Value = I Int | B Bool | Error
data Expr = Val Value | Add Expr Expr | If Expr Expr Expr
data Type = INT | BOOL | ERROR

tval :: Value -> Type
texp :: Expr -> Type
eval :: Expr -> Value

instance Ord Type where
 t ≤ t'  =  t == ERROR  ||  t == t'
```

Fig. 1. Definitions from Garby et al. [3]

(2) We add a form of possibly diverging computation to the simple expression language, via a form of unbounded search. This leads to richer typing: instead of just `BOOL` and `INT` for terms producing values of those types, we now distinguish between terms which are known to terminate or ones which might diverge. So we have types `BOOL b` and `INT b`, where boolean `b` indicates whether possible divergence (`True`) or guaranteed termination (`False`). The `FAIL` type is updated similarly, to distinguish terms that are guaranteed to fail at runtime, from those that fail or else diverge. For computations that are *guaranteed* to diverge, we add a type `LOOP`.
(3) With this more complex ordering on types, we adopt monadic definitions of the type checker and the evaluator.
(4) Using these enriched definitions, we prove the specification proposed by Garby et al. [3], in Agda. Here, this specification involves relating two monadic computations, namely the type checker and the evaluator. To do this concisely, we make use of a binary indexed monad in the proof.

The motivation for this development is to enrich the kind of information the type checker can provide. Usually (and in Garby et al. [3]), type checkers do not distinguish between situations that might lead to runtime errors, and those which must. Such a distinction would pave the way to providing new error information to programmers, when their programs do not pass the type checker. One might wish, for example, to report example inputs to a function which are guaranteed to lead to a runtime error. It is common practice to provide such information in the world of model checking, in the form of error traces [4]. Formalizing such error reporting remains, however, for future work. We here just address the issue of relating typing and evaluation with a more refined notion of untypability.

All code following is written in Agda (checked with version 2.7.0.1), making modest use of an alternative standard library for basic data structures. Readers unfamiliar with Agda will find that our code looks very similar to Haskell, except for the use of unicode and user-defined mixfix syntax, where underscores in the names of functions indicate argument positions. Here is a brief summary of other Agda features we use. Pattern-matching equations can be extended to match on an additional expression using `with`. The additional argument is listed following a `|` in subsequent equations. The library provides types $\mathbb{Z}$ of integers, $\mathbb{N}$ of unary natural numbers, and $\mathbb{B}$ of booleans, with values `tt` and `ff`. Unlike Haskell, Agda allows the same name to be used for a constructor of two different types, but the same name may not be used at both the term and type levels (because the notion of level is more complex in Agda). Finally, inputs to functions can be designated implicit

```
data Val : Set where
 I : ℤ → Val
 B : 𝔹 → Val

data Expr : Set where
 Var : Expr
 Value : Val → Expr
 Add : Expr → Expr → Expr
 IsZero : Expr → Expr
 Cond : Expr → Expr → Expr → Expr
 Search : Expr → Expr
```

Fig. 2. Values and expressions

```
data Result (V : Set) : Set where
 Value : V → Result V
 Fail : Result V
 Unfinished : Result V
```

Fig. 3. Results produced by the evaluator

using curly braces. Agda tries to infer values for those at call sites. Occasionally this fails, and then the programmer may resort to writing those arguments explicitly, in curly braces.

2 THE EXPRESSION LANGUAGE

The syntax for values and expressions is in Figure 2. Similarly to Garby et al., we work with a first-order language with integer and boolean values. Our expression syntax includes values (`Value`), additions (`Add`), and conditionals (`Cond`), just like Garby et al. We add a test for zero (`IsZero`), to have a way to compute a boolean. Most notably, we add a construct (`Search`) that searches for a natural-number value that makes an expression evaluate to zero. The value is substituted for a sole variable (`Var`) allowed by the language.

3 THE EVALUATOR

The evaluator produces one of three possible results, as shown in Figure 3. It could produce a value (`Value`), failure (`Fail`), or report that it ran out of fuel (`Unfinished`). We are using a standard simple technique for incorporating possibly diverging functions into a total programming language (here, Agda), where the function takes in an extra natural-number input, bounding the number of steps of computation. If this fuel parameter reaches zero and the computation is not complete, then our evaluator will return `Unfinished`.

To reduce boilerplate in the evaluator, we define `Result` as a monad, with the code in Figure 4. Values are propagated by `>>=`, but `Fail` and `Unfinished` terminate computation. We make use of Agda's approach to type classes using an `instance` block, to declare that `Result` is a monad.

Finally, we define the evaluator with typing

```
eval : ℕ → Expr → ℕ → Result Val
```

In call `eval g e v`, the natural `g` is the fuel, and `v` is the value for the sole variable. We follow the proposed structure of Garby et al., by having a helper function for each case of evaluation. So for evaluating an `Add` expression, we have a helper function `add`. Garby et al. give this helper the type

```
infixr 8 _>>=r_

_>>=r_ : ∀{A B : Set} → Result A → (A → Result B) → Result B
Value x >>=r r = r x
Fail >>=r r' = Fail
Unfinished >>=r r' = Unfinished

returnr : ∀{A : Set} → A → Result A
returnr = Value

instance
  ResultMonad : Monad Result
  ResultMonad = record { return = returnr ; _>>=_ = _>>=r_ }
```

Fig. 4. Monadic combinators for `Result`

`Value → Value → Value`. This simple type hides some complexity, because `Error` is included in their type for values. Here, using the monadic structure of `Result` we write:

```
add : Val → Val → Result Val
add (I x) (I y) = return (I (x +ℤ y))
add _ _ = Fail
```

Our type `Val` of values does not include errors (see Figure 2), so to return `Fail`, the result type of `add` must use `Result`. The test for zero has just as simple a helper function:

```
isZero : Val → Result Val
isZero (I x) = return (B (x =ℤ 0ℤ))
isZero _ = Fail
```

It returns a boolean value if given an integer, and otherwise fails.

Because conditional is lazy in its then- and else-parts, we write the following helper for `Cond`:

```
cond : Val → Result Val → Result Val → Result Val
cond (B x) y z = if x then y else z
cond _ _ _ = Fail
```

This takes in terms of type `Result Val` for then- and else-parts, representing that computation is not strict in those.

Deferring the code for `search` for a moment, this brings us to the definition of `eval` in Figure 5. We use do-notation (supported by Agda) to use the monadic combinators for `Result`. In the case for `IsZero`, for example, we bind the result of the recursive call `eval g e v` to `r`, which then has type `Val`. This value is given to the helper function `isZero`, which returns a value of type `Result Val`, as required by the return type of `eval`.

3.1 Implementing Search

The missing piece of the evaluator is the code to process `Search` expressions. This is given by helper function `search` in Figure 15. Some extension of Garby et al. [3] is required at this point, because on their methodology, the helper functions in the various cases of `eval` should be presented with values (`Val`), or perhaps results of computation (`Result Val`). But for `Search e`, we need to try evaluating `e` repeatedly with different values for the variable. So it seems that the helper function

```
eval : ℕ → Expr → ℕ → Result Val
eval g Var v = return (I (toℤ v))
eval g (Value x) v = return x
eval g (Add e1 e2) v =
  do
    r1 ← eval g e1 v
    r2 ← eval g e2 v
    add r1 r2
eval g (IsZero e) v =
  do
    r ← eval g e v
    isZero r
eval g (Cond e1 e2 e3) v =
  do
    b ← eval g e1 v
    cond b (eval g e2 v) (eval g e3 v)
eval g (Search e) _ = search g (eval g e)
```

Fig. 5. The evaluator

```
mutual
 search : ℕ → (ℕ → Result Val) → Result Val
 search g vv =
  do
   d ← vv g
   searchh g vv d
 searchh : ℕ → (ℕ → Result Val) → Val → Result Val
 searchh g vv (B x) = Fail
 searchh g vv (I x) with x =ℤ 0ℤ
 searchh g vv (I x) | tt = return (I (toℤ g))
 searchh zero vv (I x) | ff = Unfinished
 searchh (suc g) vv (I x) | ff = search g vv
```

Fig. 6. The helper function for searching for a zero

`search` might need to deviate from the approach in Garby et al. [3], since it cannot take just a single value or result.

We solve this problem by calling `search` with the partial application `eval g e`, which is missing the argument for the value of the sole variable of `e`. The type of this partial application is `ℕ → Result Val`, which is what the code for `search` in Figure 15 expects for its second argument `vv`. The code for `search` binds the value of `vv` on the current fuel `g`, and then calls the mutually recursive helper `searchh`. That function checks to see if the value is integer zero. If so, it returns `g` (in the `Result` monad). Otherwise, it either reports `Unfinished` if the fuel has been exhausted, or else recursively continues the searches for a zero, decrementing the fuel from `suc g` to `g` (in the last equation of Figure 15).

```
data Data : Set where
  INT : Data
  BOOL : Data

data RType : Set where
  DATA : (c : Data) → RType
  UNFINISHED : RType
  FAIL : RType

data EType (X : Set) : Set where
  DATA : (b : 𝔹)(c : X) → EType X
  FAIL : 𝔹 → EType X
  UNKNOWN : EType X
  LOOP : EType X
```

Fig. 7. The syntax of runtime types (RType) and static types (EType)

4 THE SYNTAX OF TYPES

Our aim is to define a type checker `texp` satisfying the following version of the specification of Garby et al. [3]:

```
texp e ≪ tresult (eval g e v)
```

We will need a function `tresult` to compute a type for a value of type `Result Val`, produced by `eval`. Differently from Garby et al., the relation ≪ that we use relates two different kinds of types. The type produced by `texp` has Agda type `EType Data`, representing static types. The type produced by `tresult` has Agda type `RType`, for runtime types. The definitions are in Figure 7. There is an Agda type `Data` for types common to both runtime and static. Constructors `DATA` (taking an argument of type `Data`) and `FAIL` are common to both datatypes. In the static case (`EType`), these constructors take a boolean argument indicating whether the terms might diverge (`True`) or certainly terminate (`False`). For the situation where the evaluator runs out of fuel and returns `Unfinished`, we have runtime type `UNFINISHED`. Statically, we will use a simple positivity analysis to detect if the search for a zero cannot possibly succeed. In that case, the static type is `LOOP`. Finally, static type `UNKNOWN` represents uncertainty, as generally inherent to type checking.

The type constructor `EType` is a monad, as shown by the code in Figure 8. The definition of bind (`>>=e`) for `EType` expresses that known failure (`FAIL`), uncertainty (`UNKNOWN`), and known divergence (`LOOP`) cut off type checking. This leaves the case where we are binding an `EType A` of the form `DATA b t`. This is when type checking has determined that a term has `Data` type `t`, with boolean `b` indicating possibility of divergence. In this case, we send `t` to the functional argument of bind. But we use `b` to relax the `EType` returned by `f t`. For if `b` indicates possible divergence, that possibility should be retained. This is implemented by the call `b ⇓ f t`, using the function ⇓ defined in Figure 9. If the type to be relaxed is of the form `DATA b' t` or `FAIL b'`, then the relaxation uses `b || b'` for the possibility of divergence. This means that if either the current type or the input `b` indicate that divergence is possible, then so does the resulting type. Otherwise, relaxation does not affect the type.

5 THE TYPE CHECKER

Typing of values and results is shown in Figure 10. Integer values (`I`) have type `INT`, and boolean values (`B`) `BOOL`. This is computed by `tval`. `RTypes` are computed for `Result Vals` by `tresult`, in

```
_>>=e_ : ∀{A B : Set} → EType A → (A → EType B) → EType B
DATA b t >>=e f = b ⇓ f t
FAIL b >>=e f = FAIL b
UNKNOWN >>=e f = UNKNOWN
LOOP >>=e f = LOOP

returne : ∀{A : Set} → A → EType A
returne x = DATA ff x

instance
  ETypeMonad : Monad EType
  ETypeMonad = record { return = returne ; _>>=_ = _>>=e_ }
```

Fig. 8. Monadic combinators for EType

```
_⇓_ : ∀{A : Set} → 𝔹 → EType A → EType A
b ⇓ DATA b' t = DATA (b || b') t
b ⇓ FAIL b' = FAIL (b || b')
b ⇓ t = t
```

Fig. 9. Relaxing the possibility of divergence in a type

```
tval : Val → Data
tval (I _) = INT
tval (B _) = BOOL

tresult : Result Val → RType
tresult (Value v) = DATA (tval v)
tresult Fail = FAIL
tresult Unfinished = UNFINISHED
```

Fig. 10. Typing of results and values

the obvious way: `Fail` has type `FAIL`, and `Unfinished` `UNFINISHED`. `Value` results are mapped to `DATA` types.

The type checker `texp` for expressions follows the structure proposed by Garby et al.: there is a helper function for each form of expression. Here, since our type checker is monadic, the helper functions are called with `Data` types (i.e., values of type `Data`, from Figure 7), and produce `Data` types in the `EType` monad. For `Add` and `IsZero` expressions, the helper functions are straightforward:

```
add' : Data → Data → EType Data
add' INT INT = return INT
add' _ _ = FAIL ff

isZero' : Data → EType Data
isZero' INT = return BOOL
isZero' _ = FAIL ff
```

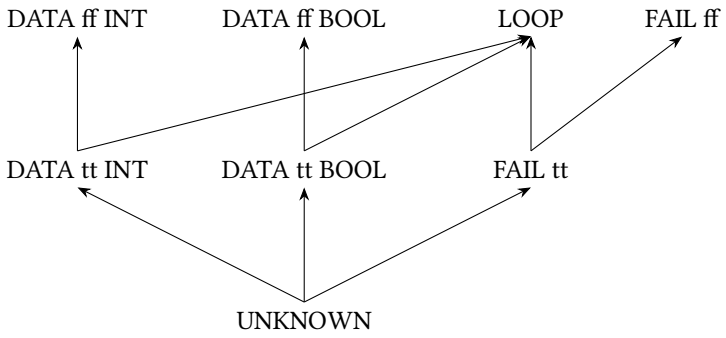

Fig. 11. The partial order on types

For conditionals (`Cond`), the helper function needs to combine the `ETypes` obtained for the then- and else-parts. We follow Garby et al. in using the order-theoretic perspective, and defining

```
cond' : Data → EType Data → EType Data → EType Data
cond' BOOL t1 t2 = t1 ⊓ t2
cond' _ _ _ = FAIL ff
```

Here, `t1 ⊓ t2` is the greatest lower bound of the `ETypes` `t1` and `t2`, in the partial order discussed next.

5.1 An Ordering on Static Types

Figure 11 shows the partial order on `ETypes`, from which `⊓` is derived. At the bottom of the order we have `UNKNOWN`, and as we proceed upwards, we gain certainty. The types `DATA tt INT`, `DATA tt BOOL`, and `FAIL tt` are one level above `UNKNOWN`. The boolean `tt` represents the possibility of divergence. So the type `DATA tt INT` is for terms which either diverge or else produce an integer value. And the type `FAIL tt` is for terms which either diverge or exhibit a runtime error, like trying to add an integer to a boolean. These types express less certainty than the similar types one level higher with boolean `ff`. For example, `DATA ff INT` is for terms which are guaranteed to terminate with an integer value. There is no uncertainty about termination. Finally, the type `LOOP` is for terms guaranteed to diverge. This again reflects greater certitude than the types like `DATA tt INT`.

This ordering is implemented in the code for `⊓`, shown in Figure 12. The last two equations use the `⇓` function of Figure 9 to relax the possibility of divergence in a type. For example, suppose we are combining `LOOP` with, say, `FAIL ff`. For conditionals, this would represent having a then-part, say, which is known to diverge, and an else-part which is known to fail. Since we do not know statically which branch will be taken, the best we can do for the type of the whole conditional is `FAIL tt`, representing the fact that the term either diverges or else exhibits a runtime error. This is what will be produced by the call `tt ⇓ (FAIL ff)`, and is indeed the greatest lower bound of those types, in the ordering of Figure 11.

5.2 Positivity analysis

The final helper function needed for the type checker `texp` is for `Search` expressions. We use this as an opportunity to consider static detection of divergence, where the type checker returns type `LOOP`. This is done here with a simple positivity analysis for expressions (much like one that forms an initial example in Garby et al. [3]). This analysis will be sound but on purpose, not complete for solvability of integer expressions. Small extensions of the language (for example, including

```
_⊓_ : EType Data → EType Data → EType Data
t ⊓ UNKNOWN = UNKNOWN
UNKNOWN ⊓ t = UNKNOWN
DATA b c ⊓ DATA b' c' = if c =c c' then DATA (b || b') c else UNKNOWN
DATA _ _ ⊓ FAIL _ = UNKNOWN
FAIL _ ⊓ DATA _ _ = UNKNOWN
FAIL b ⊓ FAIL b' = FAIL (b || b')
t ⊓ LOOP = tt ⇓ t
LOOP ⊓ t = tt ⇓ t
```

Fig. 12. The greatest lower bound of ETypes, with respect to the ordering depicted in Figure 11

multiplication and multiple variables) would make that problem undecidable, so it is in fact more illustrative of the general situation to adopt a sound but incomplete analysis. If the linear integer polynomial is strictly positive, then it has no solution. So we test for this property.

We follow the pattern established already above. We take a monadic definition for sign, to account for the approximate nature of the analysis. This is shown in Figure 13. Signs are either `Known`, containing a boolean indicating either strictly positive (`tt`) or nonnegative (`ff`), or else `Unknown`. We make `Sign` a monad by having `Unknown` cut off the computation. Figure 14 then gives the code for computing signs for values, results, and expressions. The code for `sval` matches on the form of integers in the library we are using. They are either `mkℤ zero triv` for zero, `mkℤ (suc n) tt` for a strictly positive number, or `mkℤ (suc n) ff` for a strictly negative one. We label the latter as `Unknown`, because we are deliberately keeping the analysis simple and approximate.

The code for `sexp` returns `Unknown` unless the expression uses only `Var`, `Value`, and `Add`. We use the monadic structure of `Sign` to avoid explicity considering `Unknown` in the `Add` case of `sexp`. There, if either expression has sign `tt` indicating strict positivity, then so does the `Add` expression.

5.3 Helper Function for Search

Using the type of signs from the previous section, Figure 15 gives the helper function for type-checking an expression of the form `Search e`, when `e` has the given `Data` type. The first defining equation for `search'` says that it returns `LOOP` if `e` has type `INT` and the sign is strictly positive. If the sign is anything else (second defining equation), the type is `DATA tt INT`, indicating that evaluation of `Search e` will either diverge of produce an integer. If the type for `e` is `BOOL`, then the type is `FAIL ff`, indicating that evaluation of `Search e` will definitely fail, because `e` will evaluate to a boolean. Of course, this is just the helper function for `Search e`. It could happen that the `EType` computed for `e` indicates possible or certain divergence, failure, or unknown behavior. The monad takes care of all those cases, in the code for `texp`, which is shown next.

5.4 The Definition of the Type Checker

Using the helper functions presented above, the code for the type checker `texp` is shown in Figure 16. We use the monadic structure of `EType` throughout, to abstract away handling the various modes of typing, namely failure, certain divergence, and possible divergence. The case for `Search` expressions calls `sexp` (Section **??**) to get the sign of the expression `e`, and then passes this to the helper function `search'`. The monadic approach results in clean, simple code, while still using helper functions as advocated by Garby et al. An even bigger payoff of the monadic approach will appear in the next section, where we prove their soundness property, for this richer setting.

```
data Sign (A : Set) : Set where
  Known : A → Sign A
  Unknown : Sign A

Pos : Sign 𝔹
Pos = Known tt

Nonneg : Sign 𝔹
Nonneg = Known ff

returns : ∀{A : Set} → A → Sign A
returns = Known

_>>=s_ : ∀{A B : Set} → Sign A → (A → Sign B) → Sign B
Known x >>=s f = f x
Unknown >>=s f = Unknown

instance
  SignMonad : Monad Sign
  SignMonad = record { return = returns ; _>>=_ = _>>=s_ }
```

Fig. 13. A monadic definition of signs

```
sval : Val → Sign 𝔹
sval (I (mkℤ zero triv)) = Nonneg
sval (I (mkℤ (suc n) tt)) = Pos
sval (I (mkℤ (suc n) ff)) = Unknown
sval (B x) = Unknown

sresult : Result Val → Sign 𝔹
sresult (Value v) = sval v
sresult Fail = Unknown
sresult Loop = Unknown

sexp : Expr → Sign 𝔹
sexp Var = Nonneg
sexp (Value x) = sval x
sexp (Add e e') =
  do
    s ← sexp e
    s' ← sexp e'
    return (s || s')
sexp _ = Unknown
```

Fig. 14. Signs of values (`Val`), results (`Result Val`), and expressions (`Expr`)

```
search' : Data → Sign 𝔹 → EType Data
search' INT (Known tt) = LOOP
search' INT _ = DATA tt INT
search' _ _ = FAIL ff
```

Fig. 15. Helper function for type-checking Search expressions

```
texp : Expr → EType Data
texp Var = return INT
texp (Value v) = return (tval v)
texp (Add e1 e2) =
 do
   t1 ← texp e1
   t2 ← texp e2
   add' t1 t2
texp (IsZero e) =
 do
   t ← texp e
   isZero' t
texp (Cond e1 e2 e3) =
 do
   t1 ← texp e1
   cond' t1 (texp e2) (texp e3)
texp (Search e) =
 do
   t ← texp e
   search' t (sexp e)
```

Fig. 16. The type checker texp

6 RELATING TYPING AND EVALUATION

In this section, we prove the specification proposed by Garby et al., that the static type of an expression approximates the type of its runtime value. We formulate this soundness property as:

```
texp e ≪ tresult (eval g e v)
```

where all the functions mentioned have been defined above, and ≪ is the relation between ETypes and RTypes defined in Figure 17. Let us consider the various clauses of the definition. ≪Unknown says that the static type UNKNOWN approximates any runtime type T. This clause expresses the same idea that Garby et al. proposed, as shown in the definition of ≤ in Figure 1. It allows the type checker to signal uncertainty, whenever desired. The clause ≪Data says that if the static type is DATA b T, then the runtime type may be DATA T. The critical point here is that the static type can indicate (via boolean b) that a term might diverge, and it is still acceptable if the runtime type indicates that it actually did converge to a value of type T. A similar idea is expressed in the clause ≪Fail for the FAIL type. The clause ≪Unfinished expresses that any static type is acceptable for a computation that has not yet finished. This might seem counterintuitive, because some static types indicate convergence. In fact, with our expression language, it cannot happen that a term is statically judged to be terminating, but for some smaller values of the fuel parameter to the interpreter evaluation produces Unfinished. But the specification allows that, and it could occur,

```
data _≪_ : EType Data → RType → Set where
  ≪Unknown : ∀ {T} → UNKNOWN ≪ T
  ≪Data : ∀{T}{b} → DATA b T ≪ DATA T
  ≪Fail : ∀{b} → FAIL b ≪ FAIL
  ≪Unfinished : ∀{T} → T ≪ UNFINISHED
```

Fig. 17. The relation we prove between static and runtime types

```
⇓mono : ∀{e : EType Data}{r : RType}{b : 𝔹} →
        e ≪ r →
        (b ⇓ e) ≪ r
⊓lb1 : ∀{a b : EType Data}{a' b' : RType} →
      a ≪ a' →
      b ≪ b' →
      a ⊓ b ≪ a'
⊓lb2 : ∀{a b : EType Data}{a' b' : RType} →
      a ≪ a' →
      b ≪ b' →
      a ⊓ b ≪ b'
```

Fig. 18. Several lemmas about the relation between static and runtime types

if the type checker reported solvability of polynomials, not just unsolvability. So with our language, the only case where ≪`Unfinished` can arise is when `T` is `LOOP`. This is, of course, a critical case, because it expresses the desired soundness property when the type checker reports `LOOP`: the evaluator must always return `Unfinished` in that situation, to satisfy the specification.

Several lemmas about the relation on types are listed in Figure 18. The first says that relaxing the possibility of divergence in a static type preserves the relation. The next two express the basic order-theoretic property that ⊓ computes a lower bound. These properties are also used by Garby et al. for their relation.

6.1 Monadic Structure of the Relation

The relation ≪ of Figure 17 exhibits a monadic structure, which we use below to abstract boilerplate from the proof of soundness. The combinators corresponding to monadic return and bind are shown in Figure 19. The first, `return≪`, relates the values produced by the return combinators of the `EType` and `Result` monads. The second is more complex. It aims to relate two binds:

(1) `a >>=e f` in the `EType` monad, and
(2) `a' >>=r f'` in the `Result` monad.

These can be related if first, `a ≪ tresult a'`. Second, we have a complex condition with a number of hypotheses:

- `v : Val`
- `b : 𝔹`
- `a ≡ DATA b (tval v)`
- `a' ≡ Value v`
- `DATA b (tval v) ≪ DATA (tval v)`

Providing all these, the monadic combinator then requires

- `f (tval v) ≪ tresult (f' v)`

```
return≪ : ∀{v : Val} →
          returne (tval v) ≪ tresult (returnr v)
return≪{v} = ≪Data

_>>=≪_ : ∀{a : EType Data}{a' : Result Val} →
        {f : Data → EType Data}{f' : Val → Result Val} →
         a ≪ tresult a' →
         (∀{v : Val}{b : 𝔹}
           {q : a ≡ DATA b (tval v)}
           {q' : a' ≡ Value v}
           {q'' : DATA b (tval v) ≪ DATA (tval v)} →
           f (tval v) ≪ tresult (f' v)) →
        (a >>=e f) ≪ tresult (a' >>=r f')
```

Fig. 19. Monadic structure of ≪

The essence of this requirement (by the combinator, of its caller) is to show that `f` and `f'` together preserve the relation, for all `v : Val`. The extra premises are provided by the combinator to its caller, to make this easier to show. Providing proofs that the terms `a` and `a'` referenced in the type of the first argument to the combinator are equal to these specific values (`DATA b (tval v)` and `Value v`) are justified by the situation in which this second argument to the combinator will be invoked; and also turn out to be quite helpful in using the combinator. The premise stating that these values are related by ≪ is provided by the combinator just as a convenience in the proof; it follows from the other premises and the requirement that `a ≪ tresult a'`. Note that we cannot declare that this monadic structure is an instance of `Monad`, because the interfaces are different: monads are not dependently typed, but these combinators are. So we cannot use `do`-notation with these, but as we will see next, we can still apply them directly with good effect.

6.2 The Proof of Soundness: Base Cases

We turn now to the proof of

```
texp-soundness : ∀{e : Expr}{g v : ℕ} →
                 texp e ≪ tresult (eval g e v)
```

Let us start with the base cases:

```
texp-soundness {Value x} {g} {v} = return≪
texp-soundness {Var} {g} {v} = ≪Data
```

They may both be proved using the `return` of the monadic proof combinator, although we prefer `≪Data` (which `return≪` returns) for the second, because in that case, Agda cannot infer the value for implicit argument `v`.

6.3 The Proof of Soundness: Arithmetic Cases

Next, let us consider the case for `IsZero`. We must prove

```
texp (IsZero e) ≪ tresult (eval g (IsZero e) v)
```

Unfolding the definitions of the type checker (`texp`) and evaluator (`eval`), this goal becomes

```
texp e >>=e isZero' ≪ tresult (eval g e v >>=r isZero)
```

Delightfully, this matches the conclusion of our monadic proof combinator `>>=≪`, which is

```
(a >>=e f) ≪ tresult (a' >>=r f')
```

```
case-isZero : ∀{v : Val} →
              isZero' (tval v) ≪ tresult (isZero v)
case-isZero {I x} = ≪Data
case-isZero {B x} = ≪Fail
```

Fig. 20. Lemma for the IsZero case

Even better, Agda can deduce that in order to apply that combinator, a must be instantiated to `texp e`, `f` to `isZero'`, `a'` to `eval g e v`, and `f'` to `isZero`. From the type of the combinator, this means we have two subgoals to prove, where the first is

```
texp e ≪ tresult (eval g e v)
```

But this is just what the induction hypothesis (i.e., under Curry-Howard, a recursive call to `texp-soundness` produces for `e`). So we need only prove the second premise of the combinator. The proof in Figure 20 is sufficient; it simply does not make use of the extra premises that the combinator provides (about the identity of a, etc.). They are not needed here. The two cases of the proof, for the two constructors of `Val`, are easily proved from the defining clauses for ≪. Those cases are:

(1) `isZero' (tval (I x)) ≪ tresult (isZero (I x))`
(2) `isZero' (tval (B x)) ≪ tresult (isZero (B x))`

From the definitions of `izZero'` and `isZero`, these simplify to

(1) `DATA ff BOOL ≪ DATA BOOL`, because the static and runtime types of `IsZero` expressions applied to integer values are both (forms of) the boolean type; and
(2) `FAIL ff ≪ FAIL`, because `IsZero` fails if applied to booleans.

Notice that the monadic proof combinator has taken care of all the error cases, and of propagating the possibility of divergence. So here, it suffices to reason as if the subexpression `e` is terminating.

Using the monadic proof combinator and this three-line lemma `case-isZero`, the case of `texp-soundness` for `IsZero` is very easy:

```
texp-soundness {IsZero e} {g} {v} =
  texp-soundness{e}{g}{v} >>=≪
  case-isZero
```

We just use the induction hypothesis (i.e., make a recursive call) for `e`, and bind the result to `case-iszero` using the proof combinator.

The case for `Add` expressions works just as nicely. We have this easily proven lemma, where we relate data in the case of two integer values, and failures in all others:

```
case-add : ∀{v1 v2 : Val} →
           add' (tval v1) (tval v2) ≪ tresult (add v1 v2)
case-add {I _} {I _} = ≪Data
case-add {I _} {B _} = ≪Fail
case-add {B _} {I _} = ≪Fail
case-add {B _} {B _} = ≪Fail
```

And the clause for `texp-soundness` is again very simple, using the combinator:

```
texp-soundness {Add e1 e2} {g} {v} =
  texp-soundness{e1}{g}{v} >>=≪
  texp-soundness{e2}{g}{v} >>=≪
  case-add
```

```
case-cond : ∀{v1 : Val}{r2 r3 : Result Val}{t2 t3 : EType Data} →
            t2 ≪ tresult r2 →
            t3 ≪ tresult r3 →
            cond' (tval v1) t2 t3 ≪ tresult (cond v1 r2 r3)
case-cond {I x} {r2} {r3} {t2} {t3} d1 d2 = ≪Fail
case-cond {B tt} {r2} {r3} {t2} {t3} d1 d2 = ⊓lb1 d1 d2
case-cond {B ff} {r2} {r3} {t2} {t3} d1 d2 = ⊓lb2 d1 d2
```

Fig. 21. Helper lemma for conditionals

6.4 The Proof of Soundness: Conditionals

Because evaluation of conditionals is lazy in the then- and else-parts, both the type checker and the evaluator as we saw them above were structured somewhat differently. The proof follows that structure. The case of `texp-soundness` is:

```
texp-soundness {Cond e1 e2 e3} {g} {v} =
  texp-soundness{e1}{g}{v} >>=≪ λ{v1} →
  case-cond{v1} (texp-soundness{e2}{g}{v}) (texp-soundness{e3}{g}{v})
```

We again use the proof-level bind `>>=≪` with a recursive call to `texp-soundness` for the if-part of the conditional. The helper function then accepts the results of the recursive calls for the then- and else-parts. Agda has trouble inferring some of the implicit arguments, so we handle them explicitly using `{v1}`. Figure 21 shows the definition of the helper lemma `case-cond`. We pattern match on the value `v1` for the if-part, accepting proofs `d1` and `d2` of the soundness property for the then- and else-parts of the conditional. If `v1` is an integer value, then the goal simplifies to

```
FAIL ff ≪ FAIL
```

This is proved by `≪Fail`. The other two cases are for when `v1` is `B tt` or `B ff`. In those cases, the goals simplify to

- `t2 ⊓ t3 ≪ tresult r2`
- `t2 ⊓ t3 ≪ tresult r3`

We see here the `⊓` operator, used in the definition of `cond'` (the helper function used by the type checker `texp`, for conditionals). These goals are instances of the ordering properties `⊓lb1` and `⊓lb2`, stated in Figure 18 above. So we prove those cases (in Figure 21) just using those lemmas.

6.5 Interlude: Soundness of the Positivity Analysis

Before we see the proof of soundness for `Search` expressions, we need to consider soundness of the positivity analysis `sexp`. This is formulated again in the style of Garby et al. [3]:

```
sexp-soundness : ∀{e : Expr}{g v : ℕ} →
                 sexp e ≪sign sresult (eval g e v)
```

Here, the partial order is as shown in Figure 22, and it follows the same basic idea as the ordering for types (of Figure 17): lower in the ordering corresponds to greater uncertainty. The `Unknown` type is a minimal element, and `Nonneg` is less than `Pos` (as it reflects the additional uncertainty of whether a term might evaluate to zero or else be strictly positive). The proof of `sexp-soundness` is simpler than the proof of soundness for `texp`, which we are in the middle of presenting. For `Add` expressions, it uses a similar proof-level monadic bind operator, shown in Figure 23. As for typing, this operator is for relating two monadic results:

```
(a >>=s f) ≪sign sresult (a' >>=r f')
```

```
data _≪sign_ : Sign 𝔹 → Sign 𝔹 → Set where
  ≪Unknown : ∀ {s} → Unknown ≪sign s
  ≪Refl : ∀{s} → s ≪sign s
  ≪Nonneg : Nonneg ≪sign Pos
```

Fig. 22. The ordering on signs

```
_>>=≪sign_ : ∀{a : Sign 𝔹}{a' : Result Val} →
        {f : 𝔹 → Sign 𝔹}{f' : Val → Result Val} →
         a ≪sign sresult a' →
         (∀{b : 𝔹}{v : Val}{q : a ≡ Known b}{q' : a' ≡ Value v} →
           Known b ≪sign sval v →
           f b ≪sign sresult (f' v)) →
        (a >>=s f) ≪sign sresult (a' >>=r f')
```

Fig. 23. Proof-level monadic bind, for proving soundness of sexp

Also as for typing, it requires proofs of the relation, for the components of these monadic results:

- `a ≪sign sresult a'`
- `∀{b : 𝔹}{v : Val} → Known b ≪sign sval v → f b ≪sign sresult (f' v)`

The second condition states that if the sign of `v` is known to be `b`, then the relation should hold of `f b` and `f' v`. Using this combinator results in a very simple case of `sexp-soundness` for `Add` expressions.

6.6 The Proof of Soundness: Search

Returning now to the proof of `texp-soundness`, we have the last case, namely for `Search` expressions. This is the most involved, because both the type checker and evaluator are most complicated for these expressions. The type checker uses the approximated positivity analysis `sexp`. The evaluator loops through values for the variable, searching for zero, using mutually recursive helper functions `search` and `searchh`. Recall that these functions use an input `vv : ℕ → Result Val`, to avoid having any direct dependence on expressions for the helper function. We have (from Figure 15)

- `search g vv` searches for a value less than or equal to `g` where `vv` successfully returns zero.
- `searchh g vv u` checks if `u` is zero, returning integer value `g`, and continuing the search otherwise, if enough gas remains.

Not surprisingly, the proof needs mutually recursive lemmas to match this structure. These are stated in Figure 24

These lemmas require some additional hypotheses about `vv : ℕ → Result Val`. First, with `s` the sign of the original expression `e` (where `e` is not presented to these lemmas), both lemmas need the assumption

`∀{n : ℕ} → s ≪sign sresult (vv n)`

This says the sign of the calls to `vv` is approximated by `s`. Where we invoke `case-search` in the proof of `texp-soundness`, we will instantiate `s` with `sexp e` and `vv` with `eval g e`. So the required property about the sign of `vv` becomes

`∀{n : ℕ} → sexp e ≪sign sresult (eval g e n)`

And this is exactly the conclusion of `sexp-soundness`, which we saw in the previous section.

```
mutual
 case-search : ∀ {s : Sign 𝔹}{vv : ℕ → Result Val}{g : ℕ} →
               (∀{n : ℕ} → DATA tt INT ≪ tresult (vv n)) →
               (∀{n : ℕ} → s ≪sign sresult (vv n)) →
               search' INT s ≪ tresult (search g vv)
 case-searchh : ∀ {u : Val}{s : Sign 𝔹}{vv : ℕ → Result Val}{g : ℕ}{b : 𝔹} →
               (∀{n : ℕ} → DATA b (tval u) ≪ tresult (vv n)) →
               (∀{n : ℕ} → s ≪sign sresult (vv n)) →
               s ≪sign sval u →
               search' (tval u) s ≪ tresult (searchh g vv u)
```

Fig. 24. Helper lemmas about searchh and search

The second requirement of case-search is that

```
∀{n : ℕ} → DATA tt INT ≪ tresult (vv n)
```

This says that vv only produces values of data type INT. A similar requirement is imposed by case-searchh. These properties are proved, in the body of texp-soundness, by the induction hypothesis, which applies to any value n for the sole variable. The proof of case-search uses the proof-level bind, although Agda needs more help in inferring the implicit arguments. So the code does not look as nice. Still, the combinator abstracts the reasoning about failure cases, and thus keeps the proofs shorter.

To conclude, the complete proof of soundness is listed in Figure 25, omitting only local definitions of h and h' in the final case. These do some local rewriting with the results of texp-soundness and sexp-soundness, which is needed because Agda does not support rewriting inline, but only as part of pattern matching.

7 RELATED WORK

This paper is, as extensively noted above, strongly inspired by Garby et al. [3]. For other related works: indexed monads appear in Hoare Type Theory and work on Dijkstra monads [5, 7]. They are used for dependently typed programming in the presence of effects. In contrast, this paper has demonstrated use of an indexed monad for concise relational reasoning about other monadic computations. Delaware et al. [2] do relate monadic computations, in the context of modular proofs of type safety in the presence of effects. They do not structure the proofs themselves monadically, as done here. Swierstra [8] shows how to assemble effectful interpreters using free monads. His work does not concern formalized relational reasoning about these. There is, of course, a voluminous literature on proofs of type safety, too vast to survey here. See Pierce [6] for a starting point.

8 CONCLUSION AND FUTURE WORK

We have seen how to extend the inspiring approach of Garby et al. [3] to distinguish between errors and uncertainty. The kinds of errors we consider are divergence and finite failure. We extend their example expression language with a form of unbounded search, for which we use a simple sound – but intentionally incomplete – divergence analysis. The approach depends on a more complex partial ordering of types, where total uncertainty (type UNKNOWN) is minimal, and progression in the ordering corresponds to increasing certainty. We used monads to manage the several different kinds of errors, both in typing and evaluation. To prove Garby et al.'s relational specification – that the static type approximates the runtime type of the result of the interpreter – we use a proof-level binary indexed monad. This factors out all the reasoning about errors, and results in a concise and

```
texp-soundness : ∀{e : Expr}{g v : ℕ} →
                 texp e ≪ tresult (eval g e v)
texp-soundness {Value x} {g} {v} = return≪
texp-soundness {Var} {g} {v} = return≪{v = I (toℤ v)}
texp-soundness {Add e1 e2} {g} {v} =
  texp-soundness{e1}{g}{v} >>=≪
  texp-soundness{e2}{g}{v} >>=≪
  case-add
texp-soundness {IsZero e} {g} {v} =
  texp-soundness{e}{g}{v} >>=≪
  case-isZero
texp-soundness {Cond e1 e2 e3} {g} {v} =
  texp-soundness{e1}{g}{v} >>=≪ λ{v1} →
  case-cond{v1}
    (texp-soundness{e2}{g}{v})
    (texp-soundness{e3}{g}{v})
texp-soundness {Search e} {g} {v} =
  texp-soundness{e}{g}{g} >>=≪
   λ{u}{b}{q}{q'} →
   case-searchh{u}{b = b}
      (λ{n} → h {u} q)
      (sexp-soundness{e})
      (h' q')
```

Fig. 25. Proof of soundness of the type checker with respect to the evaluator

manageable proof of soundness, both for typing and the positivity analysis (for divergence). Using the standard technique of a fuel-based interpreter, we are able to perform this relational reasoning about possibly diverging evaluation.

Probably the most pressing avenue for future work is to extend the approach of Garby et al. to higher-order functions. Here, it seems likely that the idea of relating static types of expressions with runtime types of results of evaluation will break down. This is because in the presence of higher-order functions, computing the runtime type for a functional value will no longer be simple or even, for many type systems of interest, decidable. Even if results are restricted to be first-order, the soundness property will have to be extended to higher order, to reason about intermediate steps of computation. For this, instead of relating static and runtime types, one might be better off relating static types and results of evaluation directly. This might take the approach closer to usual semantic approaches in Programming Languages, like step-indexed logical relations [1].

REFERENCES

[1] Amal J. Ahmed. 2006. Step-Indexed Syntactic Logical Relations for Recursive and Quantified Types. In *Programming Languages and Systems, 15th European Symposium on Programming, ESOP 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 27-28, 2006, Proceedings (Lecture Notes in Computer Science)*, Peter Sestoft (Ed.). Springer, 69–83. https://doi.org/10.1007/11693024_6

[2] Benjamin Delaware, Steven Keuchel, Tom Schrijvers, and Bruno C. d. S. Oliveira. 2013. Modular monadic meta-theory. In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*, Greg Morrisett and Tarmo Uustalu (Eds.). ACM, 319–330. https://doi.org/10.1145/2500365.2500587

[3] Zac Garby, Patrick Bahr, and Graham Hutton. 2025. The Calculated Typer (Functional Pearl). In *Proceedings of the 18th ACM SIGPLAN International Haskell Symposium* (Singapore, Singapore) *(Haskell '25)*. Association for Computing

Machinery, New York, NY, USA, 17–29. https://doi.org/10.1145/3759164.3759346

[4] Ranjit Jhala and Rupak Majumdar. 2009. Software model checking. *ACM Comput. Surv.* 41, 4, Article 21 (Oct. 2009), 54 pages. https://doi.org/10.1145/1592434.1592438

[5] Aleksandar Nanevski, J. Gregory Morrisett, and Lars Birkedal. 2008. Hoare type theory, polymorphism and separation. *J. Funct. Program.* 18, 5-6 (2008), 865–911. https://doi.org/10.1017/S0956796808006953

[6] Benjamin C. Pierce. 2002. *Types and programming languages.* MIT Press.

[7] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. 2013. Verifying higher-order programs with the dijkstra monad. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, Hans-Juergen Boehm and Cormac Flanagan (Eds.). ACM, 387–398. https://doi.org/10.1145/2491956.2491978

[8] Wouter Swierstra. 2008. Data Types à La Carte. *J. Funct. Program.* 18, 4 (July 2008), 423–436.